

7

Starting a Video Streaming App and Adding Authentication

Having mastered how to create engaging social apps such as WhatsApp and Instagram, it's now time to dive into the world of video streaming services. This chapter marks the beginning of our third project: a Netflix-like app. Let's call it Packtflix. Here, we will explore a different aspect of Android development, focusing on multimedia content delivery and user authentication, while continuing to build captivating user interfaces.

Our journey will begin by laying the groundwork for our streaming app. We'll start from scratch, setting up a new project and introducing you to the app's structure and modules.

Following the setup, we'll dive into one of the most critical aspects of any app: authenticating your users. In today's digital age, security and privacy are more relevant than ever, so you'll learn how to implement robust authentication mechanisms using OAuth2. This will ensure that your app's users can securely access their accounts and personal preferences.

Once our users can log in, we'll focus on presenting them with a rich selection of movies. We'll employ Jetpack Compose to create dynamic and responsive lists, showcasing the available content.

Finally, we'll delve into the details. Each movie or series in your app deserves its spotlight, and you'll create detailed screens for them using Jetpack Compose. This will provide users with all the information they need to decide what to watch next.

So, this chapter will cover the following topics:

- Creating the app's structure and modules
- Building the login screen
- Authenticating the app's users
- Creating your movie list
- Making the movie and series detail screen

Technical requirements

As in the previous chapter, you will need to have installed Android Studio (or another editor of your preference).

We are going to start a new project in this chapter, so it is not necessary to download the changes made in the previous chapter.

You will find the complete code that we are going to build throughout this chapter in this repository:

<https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-7>.

Creating the app's structure and modules

In this section, we'll lay the foundation for our Packtflix app by organizing it into feature modules. As we have seen before, by dividing the app into modules such as login, list, and playback, we can work on one feature at a time without affecting the others and speed up the build process for larger projects. Additionally, we'll set up a version catalog for our dependencies as we did before to streamline the management of libraries such as Jetpack Compose, Dagger Hilt, and Kotlin.

Let's start creating the project. In Android Studio, select **File | New | New Project...**, and choose **Empty Compose Activity**. Then, in the **New Project** panel, fill out **Name**, **Package name**, and **Save location**. For the **Minimum SDK** option, we will choose **API 29** again as it guarantees the best percentage of compatibility at the time of writing.

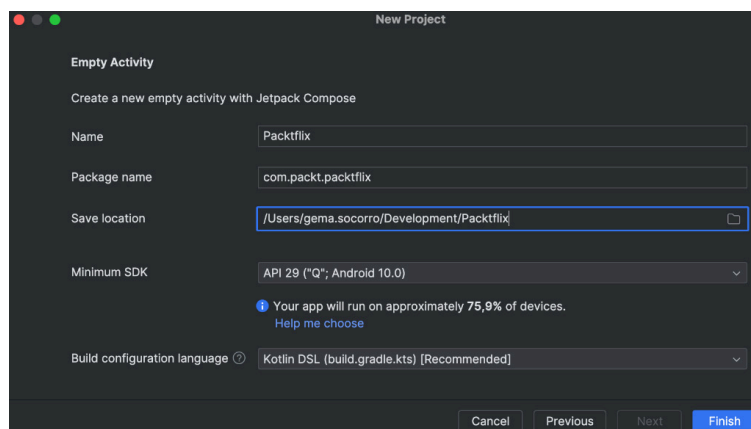


Figure 7.1: New project configuration for Packtflix

The options in *Figure 7.1* are the ones we will see using Android Studio Iguana (version 2023.2.1), though it may have variations depending on the version. For example, in other previous versions of Android Studio, we could also select whether we were going to use the version catalog for our dependencies.

Now, the version catalog is created by default, so we will already get a **libs.versions.toml** file in our project with the following content:

```
[versions]
agp = "8.3.0-alpha18"
kotlin = "1.9.0"
coreKtx = "1.12.0"
junit = "4.13.2"
junitVersion = "1.1.5"
espressoCore = "3.5.1"
```

```

lifecycleRuntimeKtx = "2.7.0"
activityCompose = "1.8.2"
composeBom = "2023.08.00"
[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", v
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeB
androidx-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-material3 = { group = "androidx.compose.material3", name = "material3" }
[plugins]
androidApplication = { id = "com.android.application", version.ref = "agp" }
jetbrainsKotlinAndroid = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }

```

This code adds the basic dependencies to the version catalog to build an app with Kotlin, Android, and Jetpack Compose.

The next step will be to create the modules needed. Here, we will create three feature modules:

- **:feature:login:** We will use this module to include the login feature
- **:feature:list:** In this module, we will include the list screen as well as the detail screen
- **:feature:playback:** In this module, we will host all the playback functionality

We will also create the following common modules:

- **:app:** This module will contain the entry point for our application
- **:common:** This module will contain common functionality needed in more than one module

To create these modules, use the **File | New | New Module...** option, as we have done in the previous projects. The final project structure should look like this:

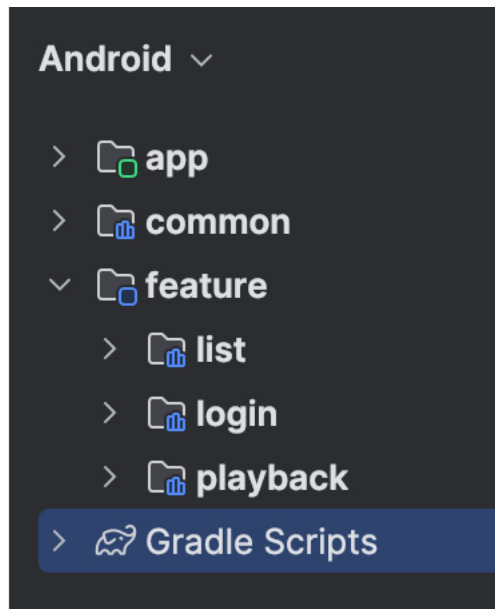


Figure 7.2: Project module structure

Now that we have created our module structure, it is time to set up the dependency injection framework.

Setting up the dependency injection framework

As we saw in the previous chapters, the need for scalability, performance optimization, and testability has made the use of a dependency injection framework practically a must in Android. In this case, we will use Hilt again (to learn more about it, please refer to [Chapter 1](#) where we did a complete review of the framework and exposed its main advantages).

Let's start adding the dependency to our version catalog. Open our **libs.versions.toml** file and add the Hilt dependencies in the **versions**, **libraries**, and **plugins** blocks, as follows:

```
[versions]
// ...
hiltVersion = "2.50"
[libraries]
// ...
androidxHilt = { module = "com.google.dagger:hilt-android", name = "hilt", version.ref = "hiltVers
hiltCompiler = { module = "com.google.dagger:hilt-android-compiler", name = "hilt-compiler", versi
[plugins]
// ...
hilt = { id = "com.google.dagger.hilt.android", version.ref = "hiltVersion" }
```

Then, we will add the plugin to the project-level **build.gradle.kts**:

```
plugins {
    ...
    alias(libs.plugins.hilt) apply false
}
```

Next, in the **build.gradle.kts** file of every module, we will have to apply the plugin and add the Hilt dependency:

```
plugins {
    //...
    alias(libs.plugins.hilt)
}
dependencies {
    //...
    implementation(libs.androidxHilt)
    kapt(libs.hiltCompiler)
}
```

Now, in the **:app** module, we can create the **PacktflixApp** class, which will be the entry point for the Hilt configuration:

```
@HiltAndroidApp
class PacktflixApp: Application() {
}
```

With this annotation, we are enabling Hilt to generate the necessary components under the hood that will be used for dependency injection throughout our application.

Finally, we should include **PacktflixApp** in **AndroidManifest.xml**, so that our app uses it instead of the default **Application** class:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android =
"http://schemas.android.com/apk/res/android"
    xmlns:tools = "http://schemas.android.com/tools">
    <application
        android:name = ".PacktflixApp"
        ...>
    ....
    </application>
</manifest>
```

Now, we are all set to start building our new project. The first step will be to build the login screen as we want our users to authenticate themselves using their credentials. Let's start working on it!

Building the login screen

To build the login screen, we will start creating a **LoginScreen** composable with Jetpack Compose. We will have to include the app's logo, fields to introduce the email and password, and a **Login** button. We can also include a text to show whether there are any errors when the user tries to log in.

This login screen is going to have four states (**Idle**, **Loading**, **Success**, and **Error**), so let's start modeling the overall **ViewState**:

```
sealed class LoginState {
    object Idle : LoginState()
    object Loading : LoginState()
    object Success : LoginState()
    data class Error(val message: String?) : LoginState()
}
```

Now, let's create the **LoginScreen** composable:

```
@Composable
fun LoginScreen() {
    val loginViewModel: LoginViewModel = hiltViewModel()
    val loginState =
        loginViewModel.loginState.collectAsState().value
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var errorMessage by remember { mutableStateOf("") }
    //...
}
```

We start the composable function by obtaining **LoginViewModel**, accessed via **hiltViewModel()**. This **ViewModel** component manages the login logic and exposes the current login state through a **StateFlow** stream. The **collectAsState().value** call converts the asynchronous stream of login states into a composable-friendly state that triggers recompositions when the login state changes.

The function uses **remember { mutableStateOf("") }** to maintain the state of user inputs for email and password within the composable's life-cycle. This state is mutable and reactive, meaning any changes to the input fields (handled by **onValueChange**) automatically update the corresponding variables and thus the UI.

Let's continue now with the next part of the composable, which will include the name of the app, the fields for **email** and **password**, and the **Login** button:

```
Surface(color = Color.Black, modifier =
    Modifier.fillMaxSize()) {
    Column(
        horizontalAlignment =
            Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center,
        modifier = Modifier
            .padding(16.dp)
    ) {
        if (loginState is LoginState.Error) {
            Text(
                text = loginState.message ?:
                    "Unknown error",
                color = Color.Red,
                modifier = Modifier
                    .padding(bottom = 16.dp)
            )
        }
    }
}
```

```

    }
    Text(
        text = "PACKTFLIX",
        color = Color.Red,
        fontSize = 36.sp,
        modifier = Modifier.padding(bottom = 32.dp)
    )
    OutlinedTextField(
        value = email,
        onValueChange = { email = it },
        label = { Text("Email") },
        colors = OutlinedTextFieldDefaults.colors(
            focusedContainerColor =
                Color.Transparent,
            focusedTextColor = Color.White,
            focusedBorderColor = Color.Gray,
            unfocusedBorderColor = Color.Gray
        ),
        modifier = Modifier.fillMaxWidth()
    )
    Spacer(modifier = Modifier.height(8.dp))
    OutlinedTextField(
        value = password,
        onValueChange = { password = it },
        label = { Text("Password") },
        visualTransformation =
            PasswordVisualTransformation(),
        colors = OutlinedTextFieldDefaults.colors(
            focusedTextColor = Color.White,
            focusedContainerColor =
                Color.Transparent,
            focusedBorderColor = Color.Gray,
            unfocusedBorderColor = Color.Gray
        ),
        keyboardActions = KeyboardActions(
            onDone = { loginViewModel.login(
                email, password) }
        ),
        modifier = Modifier.fillMaxWidth()
    )
    Spacer(modifier = Modifier.height(24.dp))
    Button(
        onClick = { loginViewModel.login(email,
            password) },
        colors = ButtonDefaults.buttonColors(
            containerColor = Color.Gray)
    ) {
        Text("Sign In", color = Color.White)
    }
    Spacer(modifier = Modifier.height(24.dp))
    if (loginState is LoginState.Loading) {
        CircularProgressIndicator()
    }
}

LaunchedEffect(loginState) {
    when (loginState) {
        is LoginState.Success -> {
            // Navigate to next screen or show success
            message
        }
    }
}

```

```

        is LoginState.Error -> {
            errorMessage = loginState.message ?:
                "An error occurred"
        }
        else -> Unit // Handle other states if
                    necessary
    }
}

```

The UI dynamically adjusts based on the current login state. For example, if the login state is **LoginState.Error**, the function renders a **Text** composable to display the error message. This conditional rendering is crucial for providing feedback to the user, such as indicating a login failure or showing a loading indicator (**CircularProgressIndicator**) when the login process is underway. This approach to UI development is declarative, with the UI's structure and content directly mapping to the application's state.

The **OutlinedTextField** composables for **email** and **password** capture user inputs, which are then used to initiate the login process (**loginViewModel.login(email, password)**) when the user clicks the **SignIn** button or completes the **password** field (via **KeyboardActions**). This demonstrates how to handle user actions and input in a composable, triggering **ViewModel** actions that ultimately lead to state changes.

Finally, the **LaunchedEffect** block listens for changes in the login state to perform side effects, such as navigation upon successful login or updating the error message state. This pattern separates side effects from the UI logic, ensuring that effects such as navigation or showing toasts only occur in response to state changes, not as a direct result of user actions.

Now, let's start working on **LoginViewModel**:

```

@HiltViewModel
class LoginViewModel @Inject constructor(
    private val loginUseCase: DoLoginUseCase
) : ViewModel() {
    private val _loginState =
        MutableStateFlow<LoginState>(LoginState.Idle)
    val loginState: StateFlow<LoginState> = _loginState
    fun login(email: String, password: String) {
        viewModelScope.launch {
            _loginState.value = LoginState.Loading
            val result = loginUseCase.doLogin(email,
                password)
            _loginState.value = when {
                result.isFailure -> LoginState.Error(
                    result.exceptionOrNull()?.message)
                else -> LoginState.Success
            }
        }
    }
}

```

Here, we start with the dependency injection stuff: the use of **@HiltViewModel** indicates that Hilt will be responsible for the instantia-

tion and provision of **LoginViewModel**. The **@Inject** constructor signifies that Hilt will inject the necessary dependencies into this **ViewModel** instance, in this case, an implementation of a use case called **DoLoginUseCase** (we will implement this use case later).

The **ViewModel** instance manages the login state using **MutableStateFlow<LoginState>**. Here, **_loginState** is a private, mutable state flow that holds the current state of the login process, which can be one of **Idle**, **Loading**, **Success**, or **Error**. The immutable **loginState** property exposes this state to the UI layer as a read-only **StateFlow**, ensuring that state updates are safely and efficiently communicated to the UI.

The login function embodies the core functionality of this **ViewModel** class. It initiates the login process by setting **_loginState** to **Loading**, indicating that the login operation has started. It then proceeds to call the **doLogin** method on the provided **loginUseCase** with the user's email and password.

After attempting to log in, the function evaluates the result. If the login attempt fails (**result.isFailure**), **_loginState** is updated to **Error** with the exception message, providing feedback on why the login failed. If the login succeeds, **_loginState** is set to **Success**, indicating a successful login process. This conditional handling ensures that the UI can react appropriately to different outcomes of the login process.

The login process is launched within **viewModelScope**, a coroutine scope tied to the **ViewModel** lifecycle. This ensures that any ongoing login operation is automatically canceled if the **ViewModel** instance is cleared (typically, when the associated UI component is destroyed), preventing memory leaks and unnecessary work.

With that, we have our login screen ready. The last step is to set up the Hilt modules and set the content of **MainActivity** to show the **LoginScreen** composable:

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            PacktflixTheme {
                LoginScreen()
            }
        }
    }
}
```

If we now execute the app, we should see the following screen:

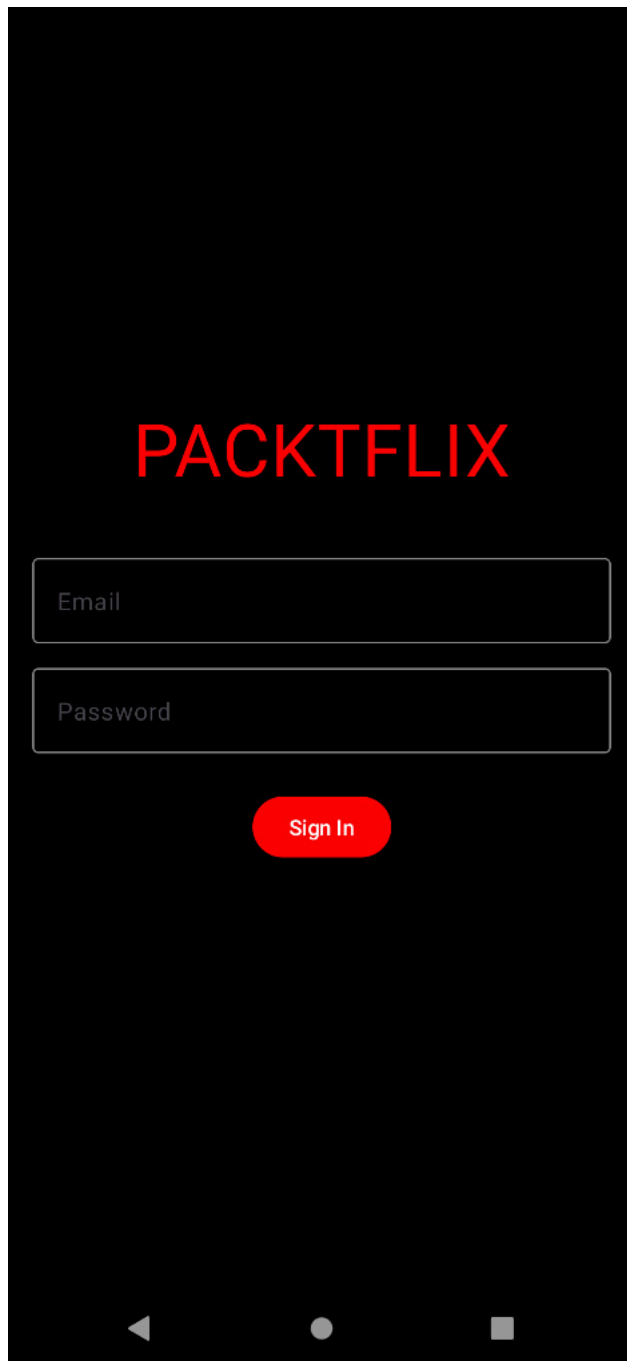


Figure 7.3: Packtflix login screen

Now that we have finished our UI, the next step will be to authenticate the users. Let's learn how to do it.

Authenticating the app's users

In mobile applications, authentication plays a critical role in protecting user data and personal information from unauthorized access. As mobile devices often serve as personal gateways to a wide array of services and store a significant amount of sensitive data, ensuring that this data is securely managed and accessed is more important than ever. One of the preferred methods for authenticating users is OAuth2.

OAuth2 is an authorization framework that allows third-party services to exchange web resources on behalf of a user. It enables users to grant websites or applications access to their information on other websites without giving them their passwords. This is particularly useful for providing functionalities such as logging in with Google, Facebook, or other social media accounts.

The following is a list of OAuth2's most important features:

- **Security:** It allows the user to authorize an application to access their resources on a different server without sharing their credentials, typically by using access tokens granted through a process involving user consent and secure token exchanges.
An OAuth **token** is a credential that represents the authorization granted to the application, allowing it to access specific resources on behalf of the user. These tokens can come in various formats, such as opaque tokens or **JSON Web Tokens (JWTs)**. Opaque tokens are simple strings without any specific structure, while JWTs are structured tokens that consist of three parts – a header, a payload, and a signature – all encoded in Base64.
- **Scalability:** It allows for the delegation of user authentication to the service that hosts the user account, by offloading the technical complexities of secure authentication and infrastructure scalability to dedicated services. These services are usually managed by specific teams in charge of the complex and resource-intensive tasks of securing and scaling authentication processes.
- **Flexibility:** It supports multiple flows (based on grant types, which will determine the flow the authentication process has to follow) for different types of clients, including mobile apps, websites, and server-side applications.
- **User Experience:** It enables a smoother login experience for users, as users can use existing accounts to sign in to new services without creating new credentials.

In essence, OAuth2 provides a secure and efficient way to implement authentication in mobile applications. It leverages existing user accounts, which simplifies the login process for users, and offloads the complexity of managing user credentials and sessions to a third-party service, enhancing both security and user experience.

Let's add this feature to our app, starting by adding the models that are needed.

Creating the user model

First, we will define a simple user model that will hold the user information we will receive upon successful authentication:

```
data class User(  
    val id: String,  
    val name: String,  
    val email: String,
```

```
// Add other fields as necessary
)
```

In this code, we are defining the basic fields needed to hold the user information (depending on the requirements of your app, these fields will be different).

Then, to build the login request that we are going to send to the backend to obtain the authentication token, we will need another data class to hold the credentials:

```
data class LoginRequest(val email: String, val password:
String)
```

Here, we are including the **email** and **password** fields, which will be mandatory to be able to log users in.

Once this request reaches the backend, if the credentials are correct, the backend will return an authorization token, which our app will store in a secure place and will use to authenticate the following API calls to the backend. We will need another model to hold this token information:

```
data class AuthToken(val token: String)
```

Now, let's set Retrofit to get this authorization token.

Using Retrofit to get the authorization token

To obtain the authorization token, we need our app to request it when the user provides their credentials. In order to send this request to the backend, we are going to use Retrofit. We already used Retrofit in [Chapter 4](#), so let's skip the introductions and start with the setup of an interface that Retrofit will use to make the HTTP requests:

```
interface AuthService {
    @POST("auth/login")
    suspend fun login(@Body loginRequest: LoginRequest):
        Response<AuthToken>
}
```

This code defines an interface called **AuthService** with a unique login function. We will pass a **LoginRequest** object with the data needed for the request and then will obtain an **AuthToken** response.

Let's build those models. First, we'll build the **LoginRequest** model:

```
data class LoginRequest(val email: String, val password:
String)
```

In this model, we will send the user's credentials – their email and password – to the backend.

Then, if the login has been successful, the backend should answer with a response including an authorization token. We will structure this response as follows:

```
data class AuthToken(val token: String)
```

This **AuthToken** model will include the aforementioned authorization token. Note that, usually, these tokens have a time window and so have to be renewed before they have expired. For simplicity, we are going to assume this token will not expire.

Now, let's create our remote data source to retrieve the authorization token:

```
class LoginRemoteDataSource(
    private val authService: AuthService
) {
    suspend fun login(email: String, password: String):
        Result<String> {
        return authService.login(
            LoginRequest(
                email = email,
                password = password
            )
        ).run {
            val token = this.body()?.token
            if (this.isSuccessful && token != null) {
                Result.success(token)
            } else {
                Result.failure(getError(this))
            }
        }
    }
}
```

Here, we define the **LoginRemoteDataSource** class, which will act as a data source layer for handling login functionality by interacting with the remote authentication service. This class will have a single dependency, **authService**, which is an interface (presumably Retrofit or a similar networking library) responsible for making network requests related to authentication. The primary function within this class, **login**, is a suspended function that takes two parameters, **email** and **password**, which are used to construct a **LoginRequest** object. This object is then passed to the **authService.login** method, initiating a network request to log the user in.

Upon receiving the response from **authService.login**, the **run** block is executed to handle the response. Inside this block, the response is checked to determine whether the request was successful (**isSuccessful**) and whether the response body contains a non-**null** token. If both conditions are met, **Result.success(token)** is returned, encapsulating the token in a successful result. This indicates that the login was successful and provides the caller with the token. Conversely, if either condition is not satisfied – meaning the request failed or the token was null – a failure re-

sult is returned by calling `Result.failure(getError(this))`. The `getError` function will analyze the `Response<AuthToken>` object to determine the nature of the failure and return an appropriate `Throwable` object that describes the error.

At this point, let's build the `getError()` function:

```
private fun getError(response: Response<AuthToken>):
    Throwable {
    return when (response.code()) {
        401 -> LoginException.AuthenticationException(
            "Invalid email or password.")
        403 -> LoginException.AccessDeniedException(
            "Access denied.")
        404 -> LoginException.NotFoundException(
            "Login endpoint not found.")
        in 500..599 -> LoginException.ServerException(
            "Server error: ${response.message()}.")
        else -> LoginException.HttpException(
            response.code(),
            "HTTP error: ${response.code()}
                ${response.message()}."
        )
    }
}
```

In this `getError` function, we map the possible values of the status code from the response to different errors. If we wanted to, we could later process those errors and show messages to the user accordingly.

Let's also define those errors, in which we will map the server response. We will define them as part of a `LoginException` sealed class, which is a special type of class in Kotlin that restricts the inheritance hierarchy to a specific set of subclasses, providing exhaustive `when` expressions and ensuring that every possible type of error is handled:

```
sealed class LoginException(loginErrorMessage: String, val
code: Int? = null) : Exception(loginErrorMessage) {
    class AuthenticationException(message: String) :
        LoginException(message)
    class AccessDeniedException(message: String) :
        LoginException(message)
    class NotFoundException(message: String) :
        LoginException(message)
    class ServerException(message: String) :
        LoginException(message)
    class HttpException(code: Int, message: String) :
        LoginException(message, code)
}
```

Now that we have our `LoginRemoteDataSource` component, it is time to define how to store the token.

Using DataStore to store the token

Introduced by Google, **DataStore** is a data storage solution that provides an efficient, secure, and asynchronous way of persisting small pieces of data. It uses Kotlin coroutines and flow streams to store data asynchronously, ensuring UI thread safety and smoother performance.

DataStore comes with several features that make it a preferable data storage option in Android applications:

- **Asynchronous by default:** DataStore operations are performed asynchronously using Kotlin coroutines, preventing blocking the main thread and improving app performance.
- **Safe and consistent:** With built-in transactional data APIs, DataStore ensures data consistency and integrity, even if an app process is killed during a write operation.
- **Type safety:** DataStore offers two implementations: Preferences DataStore, which stores and retrieves key-value pairs, and Proto DataStore, which allows for storing type-safe objects using Protocol Buffers.
- **Security:** DataStore can be integrated with encryption mechanisms to securely store sensitive information. DataStore can be combined with encryption libraries such as Tink to encrypt the data before saving, making it a more secure option for handling user credentials, tokens, and other sensitive information.

Why will we use DataStore and not Room (which we used previously for our WhatsPackt messenger project)? While both are robust data persistence libraries, they serve different purposes and have distinct use cases:

- **Use case suitability:** DataStore is designed for storing small collections of data, such as settings, preferences, or application state. It excels in handling lightweight tasks where the data structure is simple. **RoomDatabase** is a SQLite abstraction that significantly reduces the amount of boilerplate code needed to use SQLite. It's intended for more complex data storage requirements, such as storing large datasets, relational data, or when we need to perform complex queries.
- **Performance and complexity:** DataStore provides a simpler API for data storage with minimal setup, making it ideal for straightforward tasks. Its performance is optimized for small datasets and simple data structures. **RoomDatabase**, being a database, is more suited for complex queries and large datasets. It involves more setup and is heavier than DataStore but offers more features and capabilities for comprehensive data management.
- **Data security:** DataStore, especially with Proto DataStore, can easily be integrated with encryption mechanisms to store data securely, making it a more secure option for sensitive information. **RoomDatabase** supports SQLite encryption, but integrating encryption requires additional setup and possibly third-party libraries.

As we just need to store a small value (the token) and given its security features, DataStore is the best option.

So, to start using it, first, we need to set up the DataStore dependency and its version in our version catalog:

```
[versions]
datastore = "1.0.0"
[libraries]
datastore = { module = "androidx.datastore:datastore-
preferences", version.ref = "datastore" }
```

Then, need to add it to our modules' **gradle.build.kts** files:

```
dependencies {
    ...
    implementation(libs.datastore)
}
```

With this code, we are only adding it to the modules where we would need to use the dependency – initially, this will just be in the **:feature:login** module.

Now, we can start using the DataStore library. We are going to build a **LoginLocalDataSource** component, which will be responsible for storing and retrieving the token in and from the DataStore:

```
val Context.dataStore by preferencesDataStore(name = "user_preferences")
class LoginLocalDataSource(private val context: Context) {
    companion object {
        val TOKEN_KEY = stringPreferencesKey("auth_token")
    }
    suspend fun saveAuthToken(token: String) {
        context.dataStore.edit { preferences ->
            preferences[TOKEN_KEY] = token
        }
    }
    suspend fun getAuthToken(): Result<String> {
        val preferences = context.dataStore.data.first()
        val token = preferences[TOKEN_KEY]
        return if (token != null) {
            Result.success(token)
        } else {
            Result.failure(TokenNotFoundError())
        }
    }
}
class TokenNotFoundError : Throwable("Auth token not found")
```

In **LoginLocalDataSource**, first, we leverage Kotlin's property delegation feature to initialize the DataStore. By defining **val Context.dataStore** with **preferencesDataStore(name: "user_preferences")**, we ensure a single instance of the DataStore is lazily initialized and tied to the application's context. This method optimizes resource use and simplifies subsequent data operations.

Within **LoginLocalDataSource**, we define a companion object to hold **TOKEN_KEY**, a key used to store and retrieve the authentication token from the DataStore. This key is defined using **stringPreferencesKey("auth_token")**, indicating the data type we intend to store – in this case, a **String** type.

In the **saveAuthToken** function, we perform a write operation on the DataStore by calling **edit** and passing a lambda that assigns the provided token to **TOKEN_KEY**. This operation is atomic and thread-safe, ensuring the integrity of our data.

To retrieve the authentication token, **getAuthToken** also employs suspending semantics to facilitate asynchronous execution. It accesses the DataStore's data as a flow, immediately fetching the first emitted value with **.data.first()**. This operation suspends the coroutine, effectively making the data retrieval feel synchronous while maintaining the benefits of asynchronous execution. The function then checks whether the token exists and returns it wrapped in **Result<String>**, providing a straightforward way to handle success and failure. In the absence of a token, it returns **Result.failure** with a custom **TokenNotFoundError**, offering precise error handling.

Now, it is time to implement **LoginRepository**, which is responsible for coordinating between the remote and local data sources. We will build it, as always, by creating an interface in the domain layer and the implementation in the data layer. This is because the domain shouldn't have any explicit dependency from the data layer, to respect the clean architecture. So, we define the interface like so:

```
interface LoginRepository {
    suspend fun getToken(): Result<String>
    suspend fun loginWithCredentials(email: String,
        password: String): Result<Unit>
}
```

Here, the interface will have two functions: one to obtain the token so it can be used elsewhere (for example, for the backend requests to authenticate the user once it has been obtained) and another to perform the login and store the newly obtained authentication token.

Now, let's implement the repository:

```
class LoginRepositoryImpl(
    private val localDataSource: LoginLocalDataSource,
    private val remoteDataSource: LoginRemoteDataSource
): LoginRepository {
    override suspend fun getToken(): Result<String> {
        return localDataSource.getAuthToken()
    }
    override suspend fun loginWithCredentials(email:
        String, password: String): Result<Unit> {
        return remoteDataSource.login(email, password)
            .fold(
```

```

        onSuccess = {
            localDataSource.saveAuthToken(it)
            Result.success(Unit)
        },
        onFailure = {
            Result.failure(it)
        }
    )
}
}

```

The **LoginRepositoryImpl** class serves as an implementation of the **LoginRepository** interface, acting as a mediator between the application's data sources and its use cases or view models. This class abstracts the details of data retrieval and storage, providing a cohesive API for authentication processes. It relies on two primary data sources: **localDataSource** for local data storage and retrieval, and **remoteDataSource** for handling network requests related to user authentication.

In the **getToken** function, the repository directly delegates the call to **localDataSource.getAuthToken()**, which fetches the authentication token from local storage. This method returns a **Result<String>** object, encapsulating the outcome of the operation in a type-safe manner. The token retrieval is critical for checking the user's authentication status or for subsequent authenticated API calls that require a token.

The **loginWithCredentials** function implements the process of authenticating a user with their email and password. It first attempts to log in through the **remoteDataSource.login(email, password)** method. Upon a successful login, indicated by the **onSuccess** branch of the fold, it saves the received authorization token using **localDataSource.saveAuthToken(it)** and then signals the completion of the login process with **Result.success(Unit)**. Conversely, if the remote login attempt fails (**onFailure**), it propagates the failure as **Result.failure(it)**, allowing the calling code to handle the error appropriately. This design effectively separates concerns between local and remote data handling, ensuring that the repository remains the single source of truth for all authentication-related data flows within the application.

Now, we can build a use case to perform the login, consuming this **LoginRepository** component:

```

interface DoLoginUseCase {
    suspend fun doLogin(email: String, password: String):
        Result<Unit>
}

class DoLogin(
    private val loginRepository: LoginRepository
) : DoLoginUseCase {
    override suspend fun doLogin(email: String, password:
        String): Result<Unit> {
        return loginRepository.loginWithCredentials(email,
            password)
    }
}

```

```
    }  
}
```

The **DoLogin** class implements the **DoLoginUseCase** interface, encapsulating the logic required to authenticate a user by their email and password. By delegating the authentication process to **loginRepository**, it invokes **loginRepository.loginWithCredentials(email, password)** to perform the actual login operation. The **DoLogin** use case simplifies the process of user authentication into a single method call, ensuring that the details of how the login is performed are encapsulated within the repository, thereby promoting the separation of concerns and making the code easier to maintain and test.

Now, we are all set to use the login functionality. Next, let's use those tokens to validate the app requests.

Sending the authorization token in requests

To finish the users authentication:authorization token, sending in requests" authentication tasks, there is still one thing we have to do. The reason we were obtaining this authentication token was to be used in the requests the app is going to send to the backend, so it will guarantee the authenticity of the user that has generated the request. To include the token in every request, we are going to take advantage of Retrofit interceptors.

A Retrofit **interceptor** is a powerful mechanism provided by OkHttp (the underlying HTTP client used by Retrofit) that allows you to intercept and manipulate the request and response chain. Interceptors can modify requests and responses or perform actions such as logging, adding headers, handling authentication, and much more, before the request is sent to the server or after the response is received by the client.

Interceptors can be broadly categorized into two types:

- **Application interceptors:** These interceptors are called once for any single call to the server. They don't need to worry about network specifics such as retries and redirects. Application interceptors are perfect for tasks such as adding a common header to all requests, logging the request and response body for debugging purposes, or managing application-level caching.
- **Network interceptors:** These interceptors can monitor the data at the network level. They can observe and manipulate requests and responses that come from and go to the server, including any retries and redirects that occur as part of the network call process.

To add an authentication token to all outgoing requests, we will choose an application interceptor. We will choose an application interceptor in this scenario because they are designed to operate at the application layer, directly modifying requests before they are sent out and processing responses once they are received. This makes them well suited for tasks

such as adding headers that should be included in every request to the server, such as authentication tokens.

So, let's write our interceptor:

```
class AuthInterceptor(private val loginRepository:
LoginRepository) : Interceptor {
    override fun intercept(chain: Interceptor.Chain):
    Response {
        val originalRequest = chain.request()
        val token = runBlocking {
            loginRepository.getToken().getOrNull() }
        val requestWithToken = originalRequest.newBuilder()
            .apply {
                if (token != null) {
                    header("Authorization",
                        "Bearer $token")
                }
            }
            .build()
        return chain.proceed(requestWithToken)
    }
}
```

This class plays a critical role in enriching outgoing HTTP requests with authentication details. It achieves this by integrating with **LoginRepository**, from which it retrieves the current user's authorization token. Upon intercepting a request, the interceptor fetches the user's authentication: authorization token, sending in requests" token synchronously using **runBlocking** (a mechanism that allows for the seamless integration of coroutine-based asynchronous token retrieval into the synchronous flow expected by interceptors).

If a token is present, it's appended to the request as an **Authorization** header, adhering to the widely accepted bearer token format (the bearer token format is a security scheme where a client sends a token in the header of the requests to authenticate access, prefixed with the word **Bearer** followed by a space and the token itself), thereby ensuring that the request carries the necessary credentials for authentication by the server.

Using **runBlocking** within the interceptor is a pragmatic approach to accommodate the synchronous nature of the **intercept()** method, allowing for the immediate availability of the token. However, it's crucial to ensure that the token retrieval operation is efficient and non-blocking to avoid performance bottlenecks – ideally, by fetching the token from a local cache or storage.

Finally, at the end of the function, we return **chain.proceed(requestWithToken)**, which will allow Retrofit to continue processing the request, including the interceptor changes (in this case, adding the authentication header).

Now, we should include **AuthInterceptor** as an interceptor when we are building the Retrofit client:

```
@Provides
@Singleton
fun provideRetrofit(
    moshi: Moshi,
    authInterceptor: AuthInterceptor
): Retrofit {
    val okHttpClient = OkHttpClient.Builder()
        .addInterceptor(authInterceptor)
        .build()
    return Retrofit.Builder()
        .baseUrl("https://your.api.url/") // Replace
                                         with your
                                         actual
                                         base URL
        .addConverterFactory(
            MoshiConverterFactory.create(moshi))
        .client(okHttpClient)
        .build()
}
```

Here, we can see how we can integrate the interceptor we've created into our network layer setup, specifically within a Retrofit configuration.

Within the function, an **OkHttpClient** instance is created and configured to include the **authInterceptor** instance via the **addInterceptor** method. This setup ensures that every HTTP request made by this client will first pass through the **authInterceptor**, allowing it to modify the request as needed before it is sent out.

Following the configuration of the **OkHttpClient** instance, the Retrofit instance is built. The configured **OkHttpClient** instance is set as the client for Retrofit, linking the HTTP client, with its interceptor, to the Retrofit instance. Now, all the requests using this Retrofit instance will include the authentication token in the header, if it exists.

After that, we users authentication:authorization token, sending in requests” handled the app authentication, from obtaining the token and storing it to providing this token in every request. Now, it's time to build the main screen: the list of movies and series.

Creating your movie list

One of the goals of our Packtflix app is for users to have the freedom to explore and enjoy an extensive range of movies (or TV series), ensuring they stay engaged with our app. To achieve this, we must present our movie catalog in the most appealing manner possible. For that reason, in this section, we will focus on building a movie (or series!) catalog screen.

To start building the classical main screen of our streaming app, we first need to create the models we will use to represent the information.

Building the models

Start by building the **Movie** model:

```
data class Movie(  
    val id: Int,  
    val title: String,  
    val imageUrl: String,  
)
```

This is the model that will represent a movie – it includes the movie identification (**id**), its title, and a URL to an image of the movie.

Generally, movies in a streaming app are arranged by genres, so let's create a **Genre** model too:

```
data class Genre(  
    val name: String,  
    val movies: List<Movie>  
)
```

Here we defined the name of the genre (needed to render it on the screen) and a list of the movies included in that genre.

Finally, we need a **MoviesViewState** class to represent the movie list screen state:

```
data class MoviesViewState(  
    val genres: List<Genre>  
)
```

In this **MoviesViewState** class, we are including just one property, **genres**, which will store the list of genres we want to show in the list of our streaming app.

Now, we are ready to start creating the **MoviesScreen** composable.

Building the MoviesScreen composable

To build the **MoviesScreen** composable, enter the following code:

```
@Composable  
fun MoviesScreen(moviesViewState: MoviesViewState =  
    sampleMoviesScreen()) {  
    Scaffold(  
        containerColor = Color.Black,  
        topBar = { PacktflixTopBar() },  
        bottomBar = { PacktflixBottomBar() }  
    ) { innerPadding ->  
        GenreList(  
            genres = moviesViewState.genres,  
            modifier = Modifier.padding(innerPadding)  
        )  
    }
```

```
    }
}
```

As we can see, we have created our **MoviesScreen** composable and a **Scaffold** inside of it. As a **topBar** component of the **Scaffold**, we are including a new composable called **PacktflixTopBar**, then as a **bottomBar** component, we are including another new composable called **PacktflixBottomBar**. Finally, in the content of the **Scaffold**, we are showing a **GenreList** composable.

Now, let's build these three composables: **PacktflixTopBar**, **PacktflixBottomBar**, and **GenreList**.

PacktflixTopBar

Here is how we create the **PacktflixTopBar** composable:

```
@Composable
fun PacktflixTopBar() {
    TopAppBar(
        title = {
            Text(
                text = "PACKTFLIX",
                color = Color.Red,
                fontSize = 48.sp,
                modifier = Modifier.padding(bottom = 32.dp)
            )
        },
        actions = {
            IconButton(onClick =
                { /* Handle profile action */ }) {
                Icon(
                    painter = painterResource(id =
                        R.drawable.ic_profile),
                    contentDescription = "Profile"
                )
            }
            IconButton(onClick = { /* Handle more action */ }) {
                Icon(
                    painter = painterResource(id =
                        R.drawable.ic_more),
                    contentDescription = "More"
                )
            }
        },
    )
}
```

Inside **TopAppBar**, there's a title that displays the text **PACKTFLIX** on the screen – the text will be colored in red, with a large font size and some padding to create some space.

Additionally, **TopAppBar** includes two icons on the right side: the first icon is for a **Profile** button – when you tap on it, it's meant to handle a user profile-related task, although the actual function to do this hasn't been implemented yet. The second icon is for a **More** button – this is also set up to handle additional actions when clicked, but the specifics of these ac-

tions are not defined in this snippet. Each icon button has been created with an **IconButton** composable that contains an icon, and each icon gets its image from a resource file.

This is how **TopAppBar** will look:



Figure 7.4: Top bar in MoviesScreen

Let's continue with the bottom bar.

PacktflixBottomBar

Now, let's build the **PacktflixBottomBar** composable:

```
@Composable
fun PacktflixBottomBar() {
    NavigationBar (
        containerColor = Color.Black,
        contentColor = Color.White,
    ) {
        NavigationBarItem(
            icon = { Icon(Icons.Filled.Home,
                contentDescription = "Home") },
            selected = false,
            onClick = { /* Handle Home navigation */ }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Filled.Search,
                contentDescription = "Search") },
            selected = false,
            onClick = { /* Handle Search navigation */ }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Filled.ArrowDropDown,
                contentDescription = "Downloads") },
            selected = false,
            onClick = { /* Handle Downloads navigation */ }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Filled.MoreVert,
                contentDescription = "More") },
            selected = false,
            onClick = { /* Handle More navigation */ }
        )
    }
}
```

This navigation bar sports a sleek black background with icons illuminated in white, offering a stark and stylish contrast. We're also introducing four navigation items, each symbolized by a distinct icon. We've opted for icons from the Material Icons collection, assigning specific and intuitive symbols to signify **Home**, **Search**, **Downloads**, and **More** functionalities.

For each navigation item within **NavigationBarItem**, we have set up an icon along with an **onClick** listener. Initially, all these items are not selected (**selected = false**) to indicate that their selection state will be managed dynamically through user interactions or specific logic to be implemented in the future. The implementation of these sections is beyond the scope of this book.

We are also pairing each icon with **contentDescription**. This approach enhances app accessibility by offering screen readers a concise explanation of each button's function.

Once it is finished, this is how **PacktflixBottomBar** will look:



Figure 7.5: Bottom bar in MoviesScreen

Now, let's continue with the next step and complete this screen by implementing the list of movies.

GenreList

Now, let's start building the **GenreList** composable. Generally, the content of a movie screen in a streaming app is composed of a list of genres, where each one contains a list of movies. Let's use the **Genre** model we defined previously and create this list of lists. We will start creating a vertical list composed of rows where every row will show the content of every **Genre** instance:

```
@Composable
fun GenreList(genres: List<Genre>, modifier: Modifier =
Modifier) {
    LazyColumn(modifier = modifier) {
        items(genres.size) { index ->
            GenreRow(genre = genres[index])
        }
    }
}
```

To efficiently display the **GenreList** composable, we employed **LazyColumn**, chosen for its ability to render items lazily – this means it only draws the items visible on the screen, enhancing performance, especially for long lists.

Inside **LazyColumn**, we iterate over the genre list. For each genre, we call **items**, specifying the size of our genre list to determine the number of items it should prepare to display.

Then, for every item (or genre, in our context), we invoke **GenreRow**, a custom composable function that we will define in a moment. This function is responsible for rendering a single row in our list, which represents a

genre. We pass each genre to **GenreRow** by indexing it into our genres list with `genres[index]`.

Now, let's build the **GenreRow** composable that we just mentioned:

```
@Composable
fun GenreRow(genre: Genre) {
    Column(modifier = Modifier.fillMaxWidth()) {
        Text(text = genre.name, style =
            MaterialTheme.typography.headlineSmall)
        LazyRow {
            items(genre.movies.size) { index ->
                MovieCard(movie = genre.movies[index])
            }
        }
    }
}
```

We start with a vertical container, **Column**, that stretches across the full width of the screen. At the top of this container, we place the genre's name in large, readable text. This makes it clear to the user which genre they're looking at.

Right below the genre's name, we set up a horizontal scroll area, **LazyRow**, filled with movie cards. Each card represents a movie in the genre, and users can scroll through them horizontally.

For each movie in the genre, we will create a **MovieCard** composable that will show the movie thumbnail image and the name:

```
@Composable
fun MovieCard(movie: Movie) {
    Card(
        modifier = Modifier
            .padding(8.dp)
            .size(120.dp, 180.dp)
    ) {
        Image(
            painter = rememberAsyncImagePainter(model =
                movie.imageUrl),
            contentDescription = movie.title,
            contentScale = ContentScale.Crop
        )
    }
}
```

We start by using a **Card** composable that provides a Material Design card layout. This card is given specific dimensions and padding to ensure that it looks neat and uniform across the app. Specifically, we set each card to be **120dp** wide and **180dp** tall, with an **8dp** padding around it. This size is ideal for displaying movie posters without taking up too much screen space or looking too cramped.

Inside the card, we place an **Image** composable to show the movie's poster. To load the image (the movie's poster in this case) from a URL, we

use **rememberAsyncImagePainter**, a handy function that handles asynchronous image loading and caching. This means our app can fetch movie posters from the internet efficiently and display them as they become available, without blocking the UI thread.

The image is set to crop to fit the card's dimensions, ensuring that the most visually important part of the poster remains visible, even if the original image's aspect ratio doesn't exactly match the card's dimensions. This cropping also maintains a consistent appearance across all movie cards.

Finally, we include **contentDescription** for the image, using the movie's title, to make our list as accessible as possible.

With this component, we have finished our movie screen (or series screen – you just have to change the title and the content!). We can now test it using the **@Preview** annotation and providing a list of genres:

```
@Preview(showBackground = true)
@Composable
fun DefaultPreview() {
    MoviesScreenUI(moviesViewState = sampleMoviesScreen())
}
```

Here, we are using the preview feature of Jetpack Compose to see what our list will look like. We would need to create some sample content, and that's what the **sampleMoviesScreen()** function will do for us. For example, we could create this fake list of movies:

```
fun sampleMoviesScreen(): MoviesViewState {
    return MoviesViewState(
        genres = listOf(
            Genre(
                name = "Comedy",
                movies = listOf(
                    Movie(
                        id = 1,
                        title = "The Hangover",
                        imageUrl = "https://upload.wikimedia.org/wikipedia/en/b/b9/Hangoverposter0
                    ),
                    Movie(
                        id = 2,
                        title = "Superbad",
                        imageUrl = "https://upload.wikimedia.org/wikipedia/en/8/8b/Superbad_Poster
                    ),
                    Movie(
                        id = 3,
                        title = "Step Brothers",
                        imageUrl = "https://upload.wikimedia.org/wikipedia/en/d/d9/StepbrothersMP0
                    ),
                    Movie(
                        id = 4,
                        title = "Anchorman",
                        imageUrl = "https://upload.wikimedia.org/wikipedia/en/6/64/Movie_poster_An
                    )
                )
            )
        )
    )
}
```

```

    )
    ),
    Genre(
        name = "Mystery",
        movies = listOf(
            Movie(
                id = 1,
                title = "Se7en",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/6/68/Seven_%28movie%
            ),
            Movie(
                id = 2,
                title = "Zodiac",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/3/3a/Zodiac2007Poste
            ),
            Movie(
                id = 3,
                title = "Gone Girl",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/0/05/Gone_Girl_Poste
            ),
            Movie(
                id = 4,
                title = "Shutter Island",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/7/76/Shutterislandpo
            )
        )
    ),
    Genre(
        name = "Documentary",
        movies = listOf(
            Movie(
                id = 1,
                title = "March of the Penguins",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/1/19/March_of_the_pe
            ),
            Movie(
                id = 2,
                title = "Bowling for Columbine",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/e/e7/Bowling_for_col
            ),
            Movie(
                id = 3,
                title = "Blackfish",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/b/bd/BLACKFISH_Film_
            ),
            Movie(
                id = 4,
                title = "An Inconvenient Truth",
                imageUrl = "https://upload.wikimedia.org/wikipedia/en/1/19/An_Inconvenient
            )
        )
    )
)

```

Here, we are creating fake data to make the testing of **MoviesScreen** easier. Note that the URLs provided are not the actual image URLs, so you would have to replace them for actual movie posters.

Once finished, our list screen should look like this:

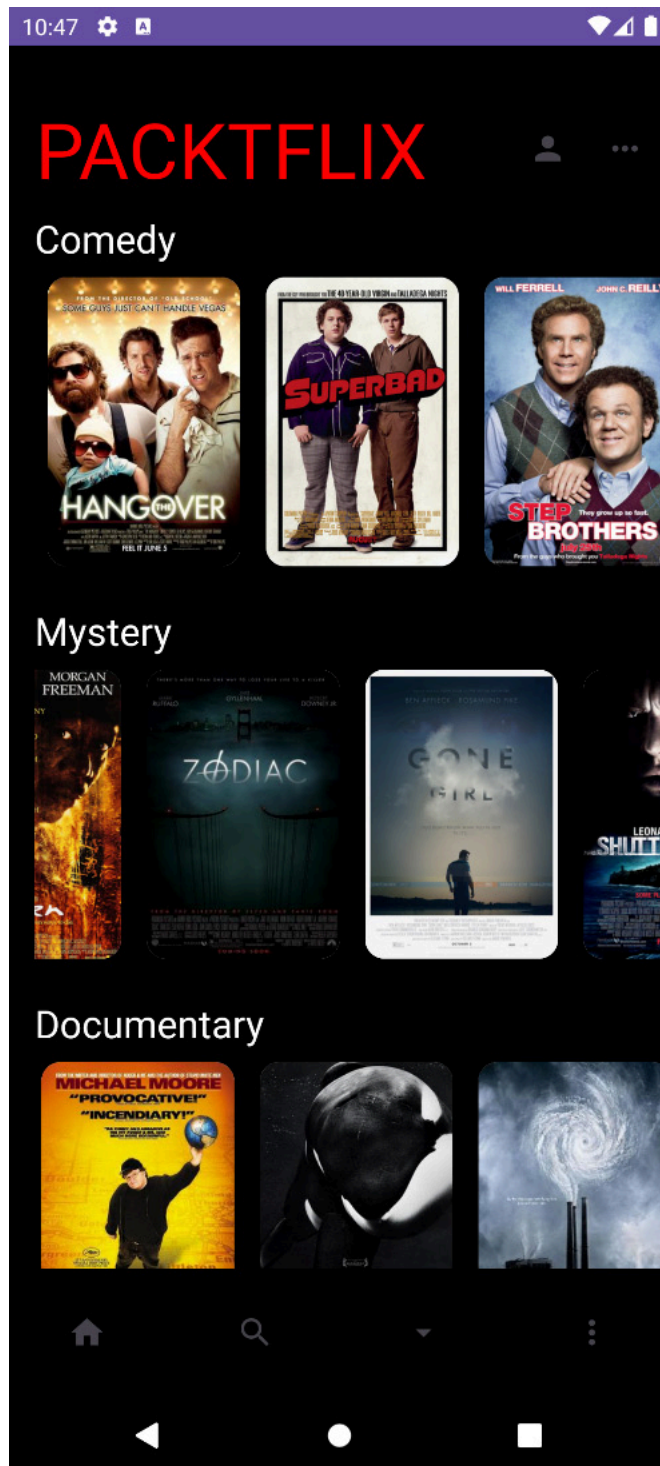


Figure 7.6: Movies list screen

Now that we have our list of genres and movies, let's build the movie (or series) details page.

Making the movie and series detail screen

In this section, we will create the detail screen, which is the screen that will be shown when the user clicks a movie or series from the list. This

screen will include information such as the plot summary, cast, year of release, and so on.

Before building the necessary composables, we need to think about the models we need. Let's start creating them.

Creating the detail models

To define the models, we need to take into account the data we want to show in the detail screen. As we would like to create the same model for both movies and series, we will build an **ItemDetail** model as follows:

```
data class ItemDetail(  
    val type: Type,  
    val title: String,  
    val imageUrl: String,  
    val rating: String,  
    val year: String,  
    val cast: List<String>,  
    val description: String,  
    val creators: List<String>,  
    val episodes: List<Episode>,  
    val movieUrl: String  
) {  
    enum class Type {  
        MOVIE, SERIES  
    }  
}
```

In the case that **ItemDetail** represents a streaming series item, we also should define the **Episode** model:

```
data class Episode(  
    val title: String,  
    val imageUrl: String,  
    val duration: String,  
    val episodeUrl: String  
)
```

Now that we have our models ready, we can start building the **DetailScreen** composable.

Building the DetailScreen

As we have done on other occasions, we will first build the structure we want the screen to have:

```
@Composable  
fun ItemDetailScreen(item: ItemDetail =  
    createFakeItemDetail()) {  
    val scrollState = rememberScrollState()  
    Column(  
        verticalArrangement = Arrangement.Top,  
        modifier = Modifier
```

```

        .fillMaxSize()
        .background(Color.Black)
        .padding(all = 8.dp)
        .verticalScroll(scrollState)
    ) {
        ItemBannerImage(item.imageUrl)
        ItemTitleAndMetadata(item.title, item.isHD,
            item.year, item.duration)
        ItemActions(item.movieUrl)
        Text(text = item.description, color = Color.Gray)
        CastAndCreatorsList(item.cast, item.creators)
        AdditionalMovieDetails(item)
    }
}

```

In **ItemDetailScreen**, all the composables included are shown in a vertical **Column**, which allows us to build the UI progressively as we add new composables.

Now, let's start building all those composables, starting with

ItemBannerImage:

```

@Composable
fun ItemBannerImage(imageUrl: String) {
    Box(modifier = Modifier.fillMaxWidth()) {
        Image(
            painter = rememberAsyncImagePainter(model =
                imageUrl),
            contentDescription = "Movie Banner",
            contentScale = ContentScale.Crop,
            modifier = Modifier
                .height(200.dp)
                .fillMaxWidth()
        )
        IconButton(
            onClick = {
                /* TODO: Handle back action */
            },
            modifier = Modifier
                .align(Alignment.TopStart)
                .padding(top = 32.dp, start = 16.dp)
        ) {
            Icon(
                imageVector = Icons.Default.ArrowBack,
                contentDescription = "Back",
                tint = Color.White
            )
        }
    }
}

```

This composable displays a banner image at the top of the screen, stretching it to fill the screen's width. It uses a **Box** composable with a **Modifier** parameter that will make sure it takes up the full width of the screen, and an **Image** composable that loads an image from a given URL with the **rememberAsyncImagePainter** function. The image is set to be 200 dp tall and automatically adjusts its width to fit the screen, ensuring that it's properly cropped to the allocated space.

On top of the image, there's an **IconButton** composable that's meant to act as a **Back** button. We place this button in the top-left corner with some padding. Inside this button, there's an icon shaped like an arrow pointing back, suggesting that pressing it should take you back to the previous screen. The icon is white to make sure it's visible on top of the banner image.

Now, let's build the **ItemTitleAndMetadata** composable:

```
@Composable
fun ItemTitleAndMetadata(
    title: String,
    isHD: Boolean,
    year: String,
    duration: String
) {
    Column {
        Text(
            text = title,
            style = MaterialTheme.typography.bodyMedium,
            fontWeight = FontWeight.Bold,
            color = Color.White
        )
        Row(verticalAlignment = Alignment.CenterVertically) {
            if (isHD) {
                Box(
                    modifier = Modifier
                        .border(BorderStroke(1.dp,
                            Color.White), shape =
                            RoundedCornerShape(4.dp))
                        .padding(horizontal = 6.dp,
                            vertical = 2.dp)
                ) {
                    Text(
                        text = "HD",
                        style =
                            MaterialTheme.typography.bodySmall,
                        color = Color.White
                    )
                }
                Spacer(modifier = Modifier.width(8.dp))
            }
            Text(
                text = year,
                style =
                    MaterialTheme.typography.bodyMedium,
                color = Color.Gray
            )
        }
        Text(
            text = duration,
            style = MaterialTheme.typography.bodyMedium,
            color = Color.Gray
        )
    }
}
```


We start by creating a **Column** layout because we want the details to stack vertically.

In this column, we will display the title of the item. The style we choose here is **bodyMedium** from the Material Theme, ensuring it fits nicely with the overall design of the app.

Next, we align our HD indicator and the year of release in a row, centering them vertically to ensure that they line up perfectly. We include a conditional check – only if **isHD** is **true** do we display an **HD** badge. We give this badge a white border and a bit of padding to make it pop against any background.

Following a small spacer, which adds some breathing room between our **HD** badge and the year, we place the text for the year. It's styled to be less prominent than the title, using a medium gray color.

Finally, below the row, we will show the duration of the item. It's also in medium gray, matching the year, and using the same **bodyMedium** style for consistency.

The next step is to create the **ItemActions** composable:

```
@Composable
fun ItemActions(
    itemUrl: String
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
    ) {
        IconButton(
            icon = Icons.Filled.PlayArrow,
            label = "Play",
            onClick = { /* TODO: Handle play action */ }
        )
        IconButton(
            icon = Icons.Default.Add,
            label = "My List",
            onClick = {
                /* TODO: Handle add to list action */ }
        )
    }
}

@Composable
fun IconButton(icon: ImageVector, label: String, onClick:
() -> Unit) {
    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.clickable(onClick = onClick)
    ) {
        Icon(
            imageVector = icon,
            contentDescription = label
        )
    }
}
```

```
        Text(text = label)
    }
}
```

We start by laying this function in a column format so that our action buttons stack vertically – this column will take up the full width available and will have padding all around for some space from the screen edges.

Inside this column, we're placing two action buttons: one for playing the item and another for adding the item to a user's personal list. To create these buttons, we are using the **ActionButton** composable function, which neatly bundles an icon and a label together into a clickable area. For the **Play** action, we are using a play arrow icon, and for adding to the list, we are using an **Add** icon.

NOTE

*We have left placeholders in the code where the play and add-to-list actions can be written. In the next chapter, we will implement the **Play** button; however, I will leave you to add the add-to-list feature yourself. To do this, one solution could be to call an endpoint when the **Add To List** button is pressed, so the backend can store it in the user list (of course, imagining that we have a backend that handles this feature). You can refer to [Chapter 4](#) where we connected Packtagram with **NewsFeed** to understand how this can be done.*

Now, let's continue with the next composable, **CastAndCreatorsList**:

```
@Composable
fun CastAndCreatorsList(cast: List<String>, creators:
List<String>) {
    Column(modifier = Modifier.fillMaxWidth()) {
        Text(
            text = "Cast",
            style = MaterialTheme.typography.titleSmall,
            color = Color.White,
            modifier = Modifier.padding(horizontal = 16.dp,
                vertical = 8.dp)
        )
        LazyRow(
            contentPadding = PaddingValues(horizontal =
                16.dp),
            horizontalArrangement =
                Arrangement.spacedBy(8.dp)
        ) {
            items(cast) { actorName ->
                Text(
                    text = actorName,
                    style =
                        MaterialTheme.typography.bodyMedium,
                    color = Color.White,
                    modifier = Modifier.background(
                        color = Color.DarkGray,
                        shape = RoundedCornerShape(4.dp)
                    ).padding(horizontal = 8.dp,
                        vertical = 4.dp)
                )
            }
        }
    }
}
```

```

    }
}
Spacer(modifier = Modifier.height(16.dp))
Text(
    text = "Created by",
    style = MaterialTheme.typography.titleMedium,
    color = Color.White,
    modifier = Modifier.padding(horizontal = 16.dp,
        vertical = 8.dp)
)
LazyRow(
    contentPadding = PaddingValues(horizontal =
        16.dp),
    horizontalArrangement =
        Arrangement.spacedBy(8.dp)
) {
    items(creators) { creatorName ->
        Text(
            text = creatorName,
            style =
                MaterialTheme.typography.bodyMedium,
            color = Color.White,
            modifier = Modifier.background(
                color = Color.DarkGray,
                shape = RoundedCornerShape(4.dp)
            ).padding(horizontal = 8.dp,
                vertical = 4.dp)
        )
    }
}
}
}
}

```

We start with **Column**, which is going to stack our elements vertically. We want this to take up the full width available, so we use **Modifier.fillMaxWidth()**.

Then, we put a header labeled **Cast** at the top. We style this text to make it stand out using **MaterialTheme.typography.titleSmall** and set the color to white. To give it some breathing room, we add padding around it.

Next, we introduce a **LazyRow** composable to display each actor's name from the cast list using a **Text** composable. We style the names to stand out against the background by applying **MaterialTheme.typography.bodyMedium** and setting the text color to white. To further distinguish each name, we give them a tag-like appearance with a dark gray background and rounded corners using **RoundedCornerShape(4.dp)**. Additionally, we add padding around the text to ensure that it doesn't touch the edges of its gray backdrop, enhancing readability and visual appeal.

Then, we separate the cast from the creators with a **Spacer** composable. This just adds a bit of vertical space between the two sections, so they don't run into each other.

For the creators, the setup is pretty much the same. We have a header labeled **"Created by"**, styled similarly to the **Cast** header but a bit larger

using **titleMedium**. Then, we list out the creators in another **LazyRow**, giving them the same styled text tags as the cast.

Now, it's time to work on the last composable of the screen,

AdditionalMovieDetails:

```
@Composable
fun AdditionalMovieDetails(item: ItemDetail) {
    Column(modifier = Modifier.fillMaxWidth()) {
        // Assuming item.episodes is a list of episodes
        // with their details
        item.episodes.forEach { episode ->
            EpisodeItem(episode = episode)
        }
    }
}

@Composable
fun EpisodeItem(episode: Episode) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable {
                /* TODO: Handle episode playback */
            }
            .padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        // Episode image
        Image(
            painter = rememberAsyncImagePainter(model =
                episode.imageUrl),
            contentDescription = "Episode Thumbnail",
            modifier = Modifier
                .size(width = 120.dp, height = 68.dp)
                .clip(RoundedCornerShape(4.dp)),
            contentScale = ContentScale.Crop
        )
        // Space between image and text details
        Spacer(modifier = Modifier.width(16.dp))
        // Episode title and duration
        Column {
            Text(
                text = episode.title,
                style =
                    MaterialTheme.typography.bodyMedium,
                color = Color.White
            )
            Text(
                text = "Duration: ${episode.duration}",
                style = MaterialTheme.typography.bodySmall,
                color = Color.Gray
            )
        }
    }
    Divider(color = Color.Gray, thickness = 0.5.dp)
}
```

In **AdditionalMovieDetails**, we're setting up a column that expands to the maximum width of its parent container. Inside this column, we're go-

ing through each episode in the `item.episodes` list and, for each one, we're calling the `EpisodeItem` composable to render the details of that episode.

Now, moving on to the `EpisodeItem` composable function, this is where we lay out each episode's information. We create a row that stretches across the full width, which can be tapped – this is where we will want to add the code for what happens when someone clicks to play the episode. We are also adding some padding for spacing.

Within this row, the first thing is the episode image. We use `rememberAsyncImagePainter` to load the image from the episode URL, and we make sure it's nicely rounded and cropped to fit a specific size. This image will act as a thumbnail for the episode.

Next to the image, we add a spacer to give some breathing room before the text details of the episode. This is followed by a column that holds two pieces of text: the episode's title, which stands out more, and below it, the duration of the episode in a smaller and less prominent color.

Lastly, after each episode item, we draw a thin gray line, a divider, to visually separate the episodes from one another. It's a common design pattern that helps users distinguish between different pieces of content.

And with this composable, we have finished the detail screen and this chapter. Our detail screen should look like this:

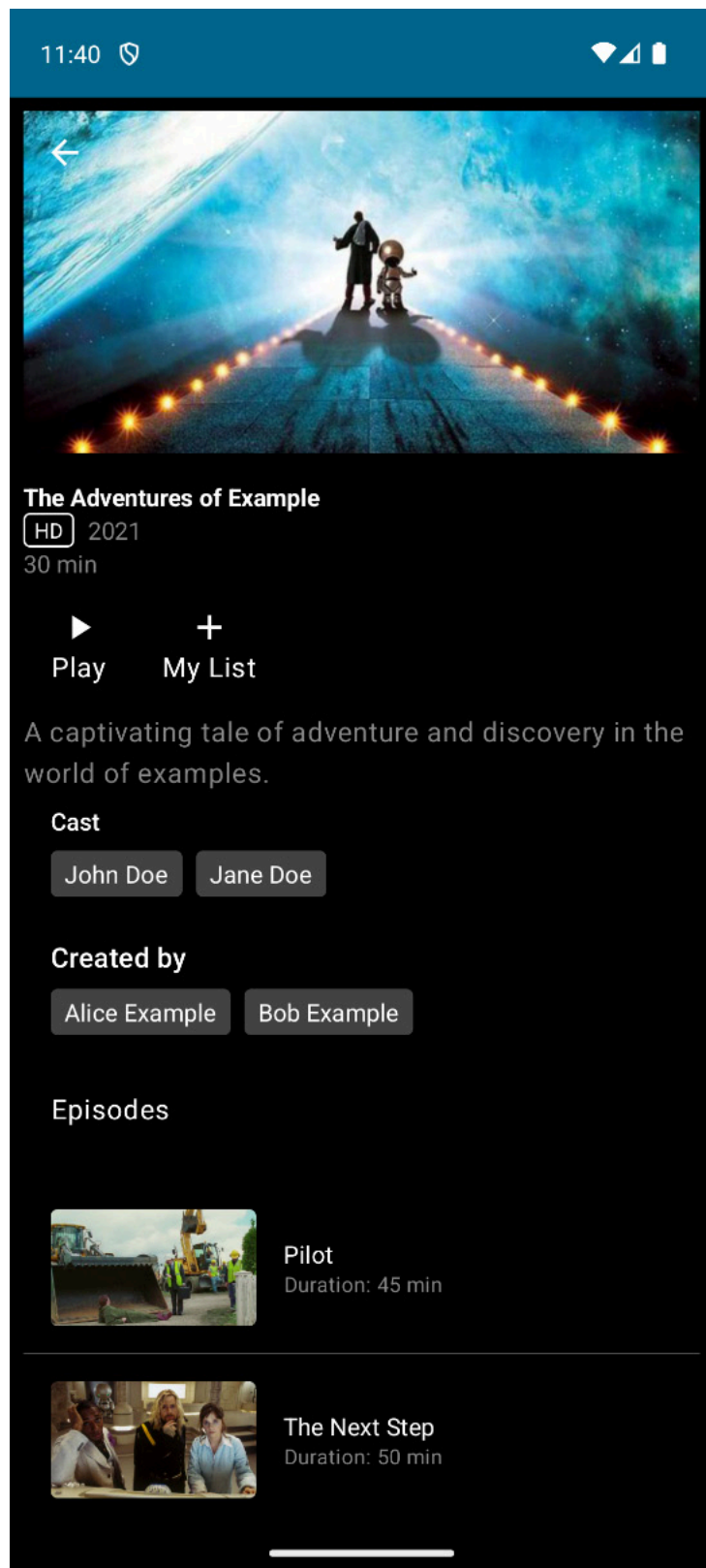


Figure 7.7: Detail screen

In the next chapter, we will bring those movies and series to life by implementing the playback.

Summary

As we close this chapter, we have laid a solid foundation for Packtflix, our video streaming app. We began by conceptualizing the project's structure and modules, setting the stage for an organized and scalable app. This structure is pivotal for our journey ahead, where complexity will grow as we add more features.

We then created the login screen, before venturing into the world of user authentication. Through the integration of OAuth2, we've equipped Packtflix with a secure authentication system that respects user privacy and guards against unauthorized access, ensuring a trustworthy environment for our users to enjoy their favorite content.

Our progress continued as we crafted a UI to display a curated list of movies, leveraging the power of Jetpack Compose to create a dynamic and engaging experience. This attention to detail in presenting content is what will turn first-time users into loyal fans of Packtflix.

In the next chapter, we will learn more about how to implement the playback, so our users can not only see the movies and series information but also play their videos.